KENNESAW STATE
UNIVERSITY

CS 4277
Optimization

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

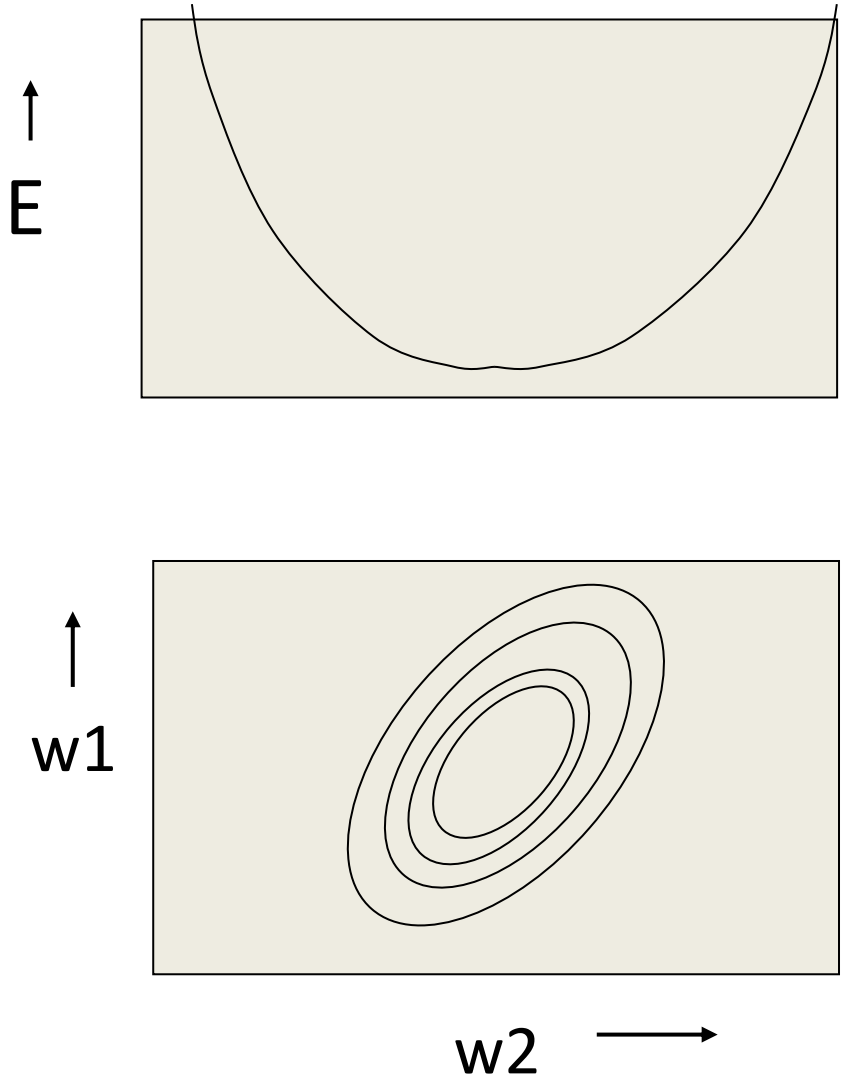
Learning Objectives

- Learning error
- Stochastic gradient descent
- Mini batch gradient descent
- Speeding up mini-batch gradient descent
  - Momentum
  - AdaGrad
  - Rmsprop
  - Adam

Feedforward networks

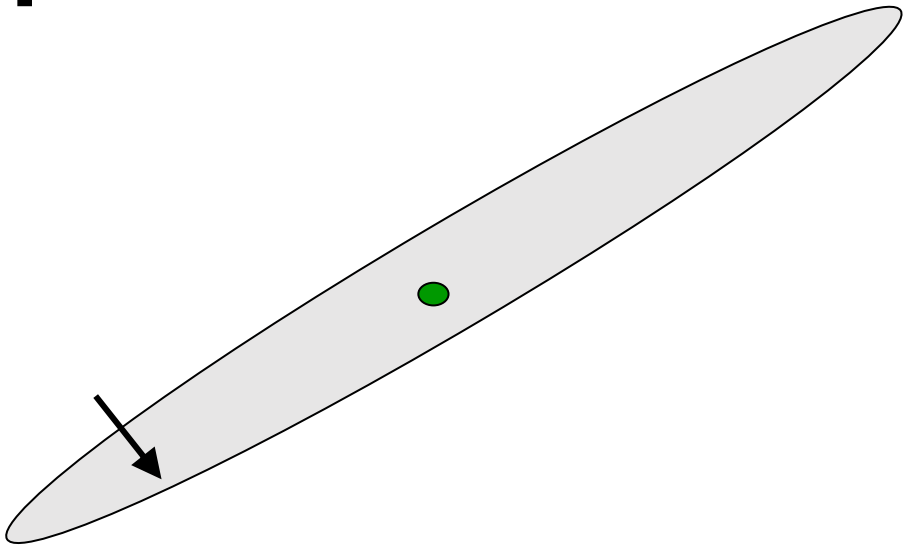
The error surface for a linear neuron

- The error surface lies in a space with a horizontal axis for each weight and one vertical axis for the error.
  - For a linear neuron with a squared error, it is a quadratic bowl.
  - Vertical cross-sections are parabolas.
  - Horizontal cross-sections are ellipses.
- For multi-layer, non-linear nets the error surface is much more complicated.
  - But locally, a piece of a quadratic bowl is usually a very good approximation.



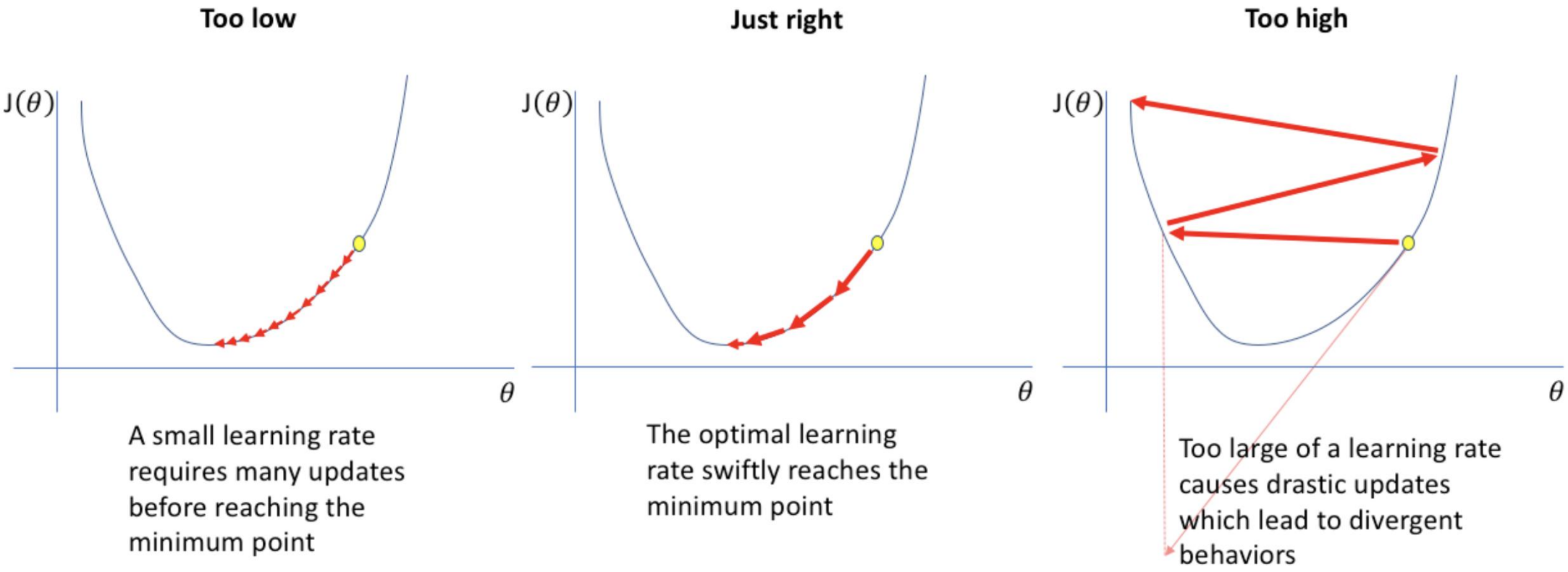
Convergence speed of full batch learning when the error surface is a quadratic bowl

- Going downhill reduces the error, but the direction of steepest descent does not point at the minimum unless the ellipse is a circle.
  - The gradient is big in the direction in which we only want to travel a small distance.
  - The gradient is small in the direction in which we want to travel a large distance.

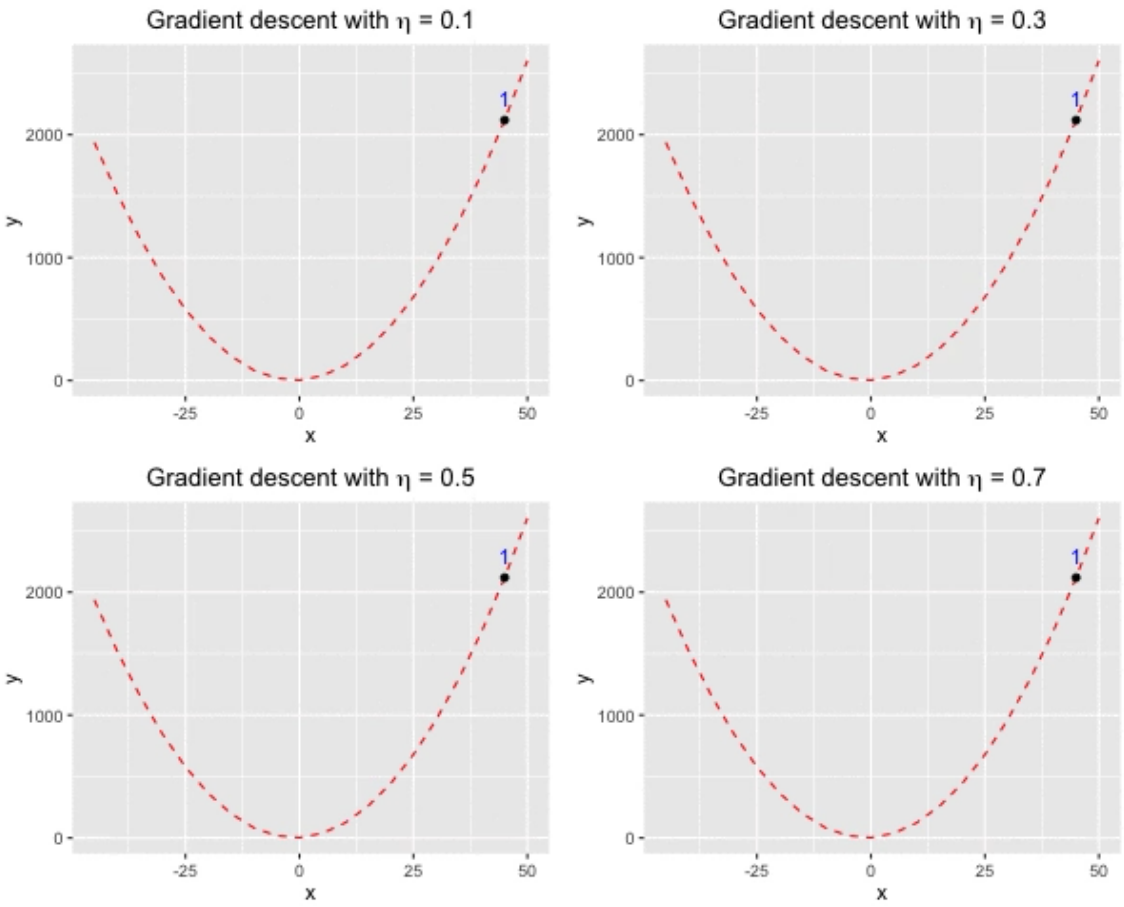


Even for non-linear multi-layer nets, the error surface is locally quadratic, so the same speed issues apply.

Recap: Learning Rate

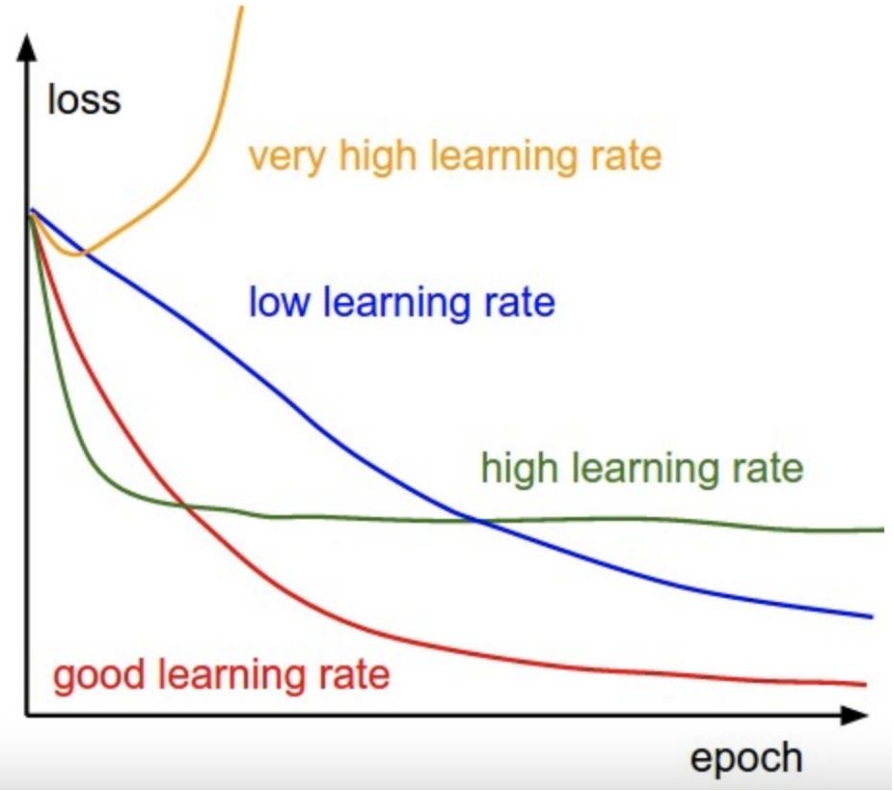


Recap: Learning Rate



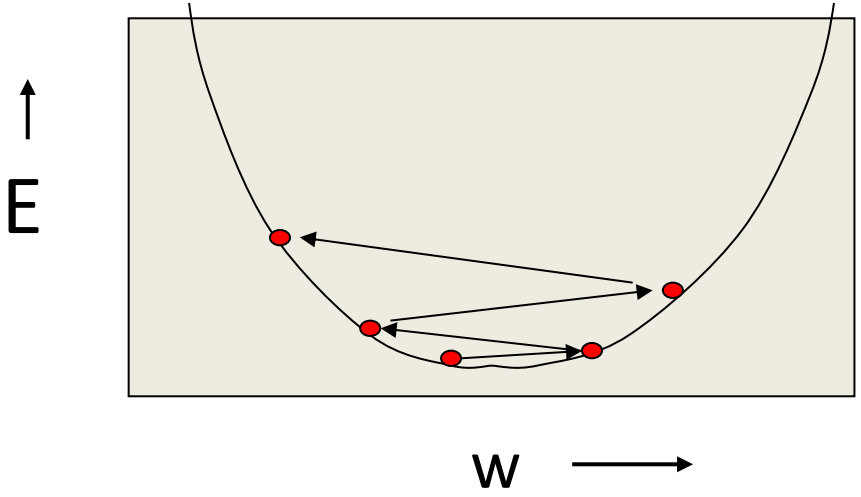
Recap: Learning Rate

- Learning rate is optimal, model converges to the minimum
- Learning rate is too small, it takes more time but converges to the minimum
- Learning rate is higher than the optimal value, it overshoots but converges
- Learning rate is very large, it overshoots and diverges, moves away from the minima, performance decreases on learning



How the learning goes wrong

- If the learning rate is too large, the weights oscillate across the ravine.
  - If the learning rate is too big, this oscillation diverges.
- What we would like to achieve:
  - Move quickly in directions with small but consistent gradients.
  - Move slowly in directions with big but inconsistent gradients.



Stochastic gradient descent

- If the dataset is highly redundant, the gradient on the first half is almost identical to the gradient on the second half.
  - So instead of computing the full gradient, update the weights using the gradient on the first half and then get a gradient for the new weights on the second half.
  - The extreme version of this approach updates weights after each case. It called “online”.
- Mini-batches are usually better than online.
  - Less computation is used updating the weights.
  - Computing the gradient for many cases simultaneously uses matrix-matrix multiplies which are very efficient, especially on GPUs
- Mini-batches need to be balanced for classes

Two types of learning algorithm

- If we use the full gradient computed from all the training cases, there are many clever ways to speed up learning.
  - The optimization community has studied the general problem of optimizing smooth non-linear functions for many years.
  - However, multilayer neural nets are not typical of the problems they study so their methods may need a lot of adaptation.
- For large neural networks with very large and highly redundant training sets, it is nearly always best to use mini-batch learning.
  - The mini-batches may need to be quite big when adapting fancy methods.
  - Big mini-batches are more computationally efficient.

A basic mini-batch gradient descent algorithm

- Guess an initial learning rate.
  - If the error keeps getting worse or oscillates wildly, reduce the learning rate.
  - If the error is falling fairly consistently but slowly, increase the learning rate.
- Write a simple program to automate this way of adjusting the learning rate.
- Towards the end of mini-batch learning it nearly always helps to turn down the learning rate.
  - This removes fluctuations in the final weights caused by the variations between mini-batches.
- Turn down the learning rate when the error stops decreasing.
  - Use the error on a separate validation set

A bag of tricks for mini-batch gradient descent

Initializing the weights

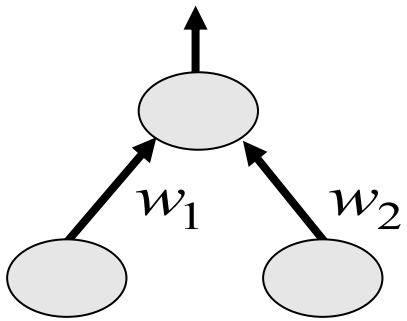
- If two hidden units have exactly the same bias and exactly the same incoming and outgoing weights, they will always get exactly the same gradient.
  - So, they can never learn to be different features.
  - We break symmetry by initializing the weights to have small random values.
- If a hidden unit has a big fan-in (number of input), small changes on many of its incoming weights can cause the learning to overshoot.
  - We generally want smaller incoming weights when the fan-in is big, so initialize the weights to be proportional to sqrt(fan-in).
- We can also scale the learning rate the same way.

Xavier Initialization

- Aim:
  - Maintain variance of activations approximately constant across layers.
  - Prevent vanishing or exploding gradient problems during training.
- Method:
  - Weights sampled from Gaussian distribution with zero mean.
  - Variance: $Var(W) = \frac{2}{n_{in}+n_{out}}$, then $W \sim N(0, \frac{2}{n_{in}+n_{out}})$
  - n_{in}: the number of neurons in the input layer
  - n_{out}: the number of neurons in the output layer
- Results:
  - Xavier initialization helps improve training performance and convergence.

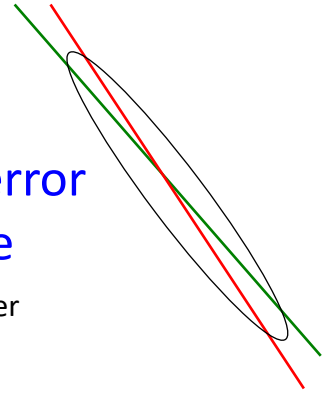
Shifting the inputs

- When using steepest descent, shifting the input values makes a big difference.
  - It usually helps to transform each component of the input vector so that it has zero mean over the whole training set.
- The hypberbolic tangent (which is 2*logistic -1) produces hidden activations that are roughly zero mean.
  - In this respect its better than the logistic.



color indicates training case

Hard to fine-tune weights



101, 101 → 2

101, 99 → 0

gives error surface

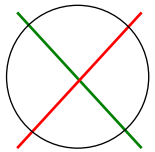
Input is very big, and they are close to each other

$-100, -100$

1, 1 → 2

1, -1 → 0

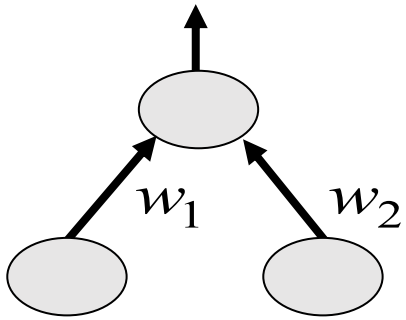
gives error surface



Easy to fine-tune weights

Scaling the inputs

- When using steepest descent, scaling the input values makes a big difference.
  - It usually helps to transform each component of the input vector so that it has unit variance over the whole training set.

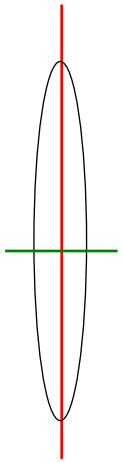


color indicates weight axis

Hard to fine-tune weights

0.1, 10 → 2
0.1, -10 → 0

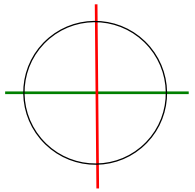
gives error surface



$\times 10, \div 10$

1, 1 → 2
1, -1 → 0

gives error surface



Easy to fine-tune weights

A more thorough method: Decorrelate the input components

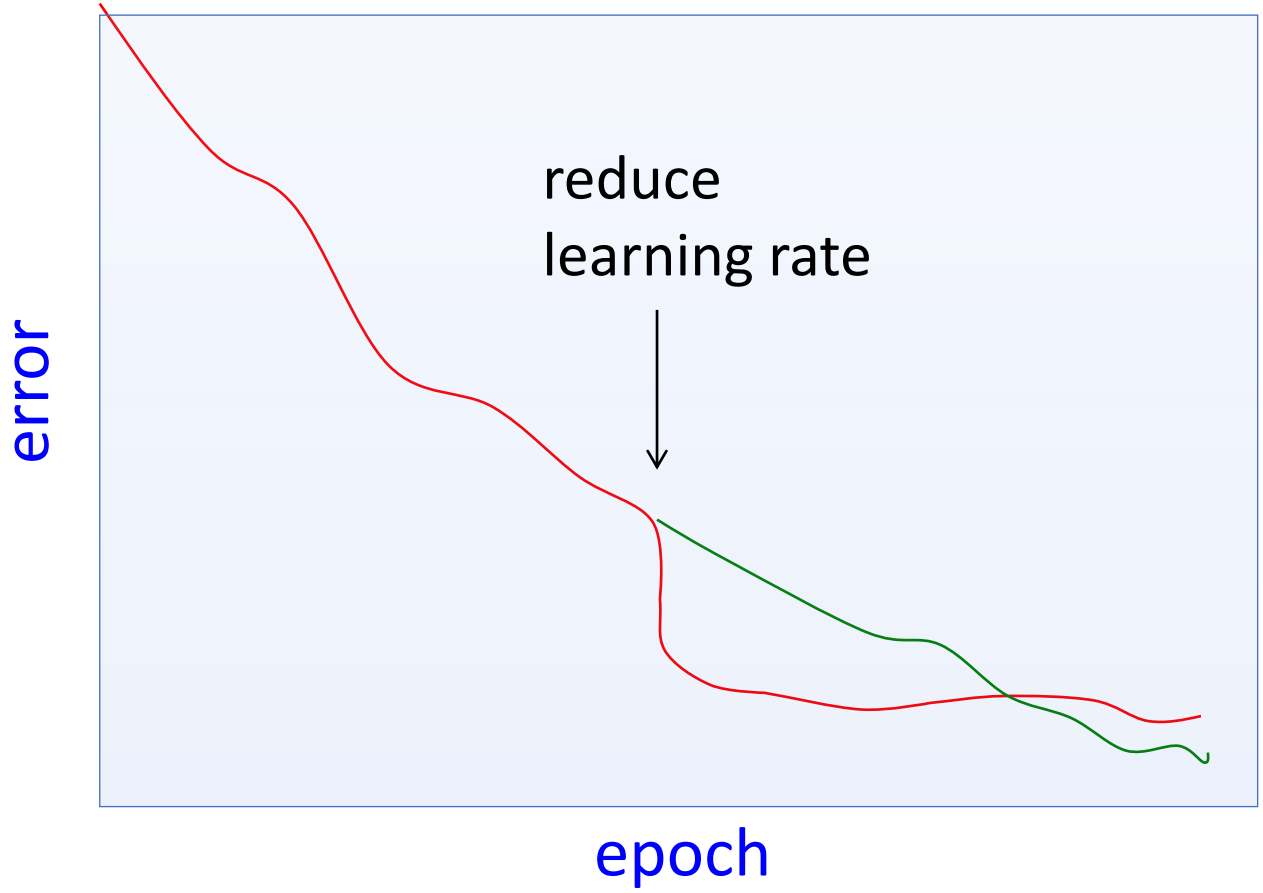
- For a linear neuron, we get a big win by decorrelating each component of the input from the other input components.
- There are several different ways to decorrelate inputs. A reasonable method is to use Principal Components Analysis.
  - Drop the principal components with the smallest eigenvalues.
    - This achieves some dimensionality reduction.
  - Divide the remaining principal components by the square roots of their eigenvalues. For a linear neuron, this converts an axis aligned elliptical error surface into a circular one.
- For a circular error surface, the gradient points straight towards the minimum.

Common problems that occur in multilayer networks

- If we start with a very big learning rate, the weights of each hidden unit will all become very big and positive or very big and negative.
  - The error derivatives for the hidden units will all become tiny and the error will not decrease.
  - This is usually a plateau, but people often mistake it for a local minimum.
- In classification networks that use a squared error or a cross-entropy error, the best guessing strategy is to make each output unit always produce an output equal to the proportion of time it should be a 1.
  - The network finds this strategy quickly and may take a long time to improve on it by making use of the input.
  - This is another plateau that looks like a local minimum.

Be careful about turning down the learning rate

- Turning down the learning rate reduces the random fluctuations in the error due to the different gradients on different mini-batches.
  - So we get a quick win.
  - But then we get slower learning.
- Don't turn down the learning rate too soon!



Speed up mini-batch learning

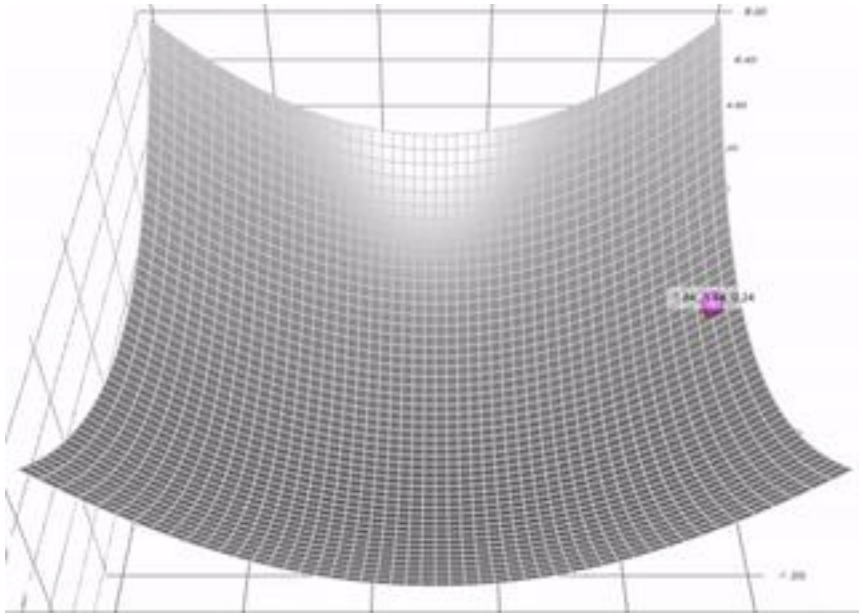
Four ways to speed up mini-batch learning

- Use "momentum"
  - Instead of using the gradient to change the position of the weight "particle", use it to change the velocity.
- Use separate adaptive learning rates for each parameter
  - Slowly adjust the rate using the consistency of the gradient for that parameter.
- RMSprop: Divide the learning rate for a weight by a running average of the magnitudes of recent gradients for that weight.
  - This is the mini-batch version of just using the sign of the gradient.
- Take a fancy method from the optimization literature that makes use of curvature information (not this lecture)
  - Adapt it to work for neural nets
  - Adapt it to work for mini-batches.

Momentum

Momentum

- Imagine rolling down a ball inside of a bowl.
- Instead of stopping at the bottom, the momentum it has accumulated pushes it forward, and the ball keeps rolling back and forth.
- In physics, momentum = mass*velocity
- In optimization: momentum = velocity



Momentum

- The momentum algorithm introduces a variable v that plays the role of velocity

$$v(t) \leftarrow \alpha v(t-1) - \epsilon \frac{\partial E}{\partial w}(t)$$

- $\alpha \in [0, 1)$, a hyperparameter determines how quickly the contributions of previous gradients exponentially decay.
- The effect of the gradient is to increment the previous velocity. The velocity also decays by α which is slightly less then 1.

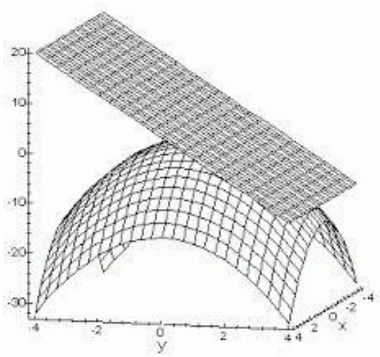
$$\Delta w(t) = v(t) = \alpha v(t-1) - \epsilon \frac{\partial E}{\partial w}(t) = \alpha w(t-1) - \epsilon \frac{\partial E}{\partial w}(t)$$

- The weight change is equal to the current velocity
- The weight change can be expressed in terms of the previous weight change and the current gradient.

The behavior of the momentum method

- If the error surface is a tilted plane, the ball reaches a terminal velocity.
  - If the momentum is close to 1, this is much faster than simple gradient descent.

$$v(\infty) = \frac{1}{1-\alpha}\left(-\epsilon \frac{\partial E}{\partial w}\right)$$



- At the beginning of learning there may be very large gradients.
  - So it pays to use a small momentum (e.g. 0.5).
  - Once the large gradients have disappeared and the weights are stuck in a ravine the momentum can be smoothly raised to its final value (e.g. 0.9 or even 0.99)
- This allows us to learn at a rate that would cause divergent oscillations without the momentum.

A better type of momentum (Nesterov 1983)

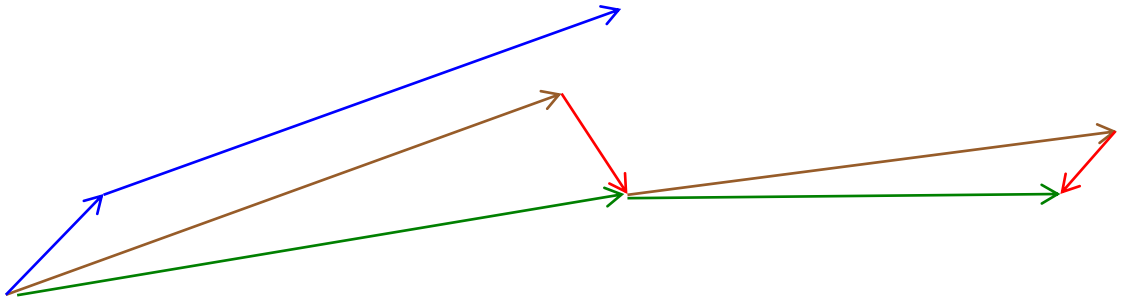
- The standard momentum method first computes the gradient at the current location and then takes a big jump in the direction of the updated accumulated gradient.
- Ilya Sutskever (2012 unpublished) suggested a new form of momentum that often works better.
  - Inspired by the Nesterov method for optimizing convex functions.

- First make a big jump in the direction of the previous accumulated gradient.
- Then measure the gradient where you end up and make a correction.
- Its better to correct a mistake after you have made it!

- The difference between Nesterov momentum and standard momentum is where the gradient is evaluated

A picture of the Nesterov method

- First make a big jump in the direction of the previous accumulated gradient.
- Then measure the gradient where you end up and make a correction.



brown vector = jump, red vector = correction, green vector = accumulated gradient

blue vectors = standard momentum

momentum vs. Nesterov momentum

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ, momentum parameter α.
Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$.
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient estimate: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
  end while

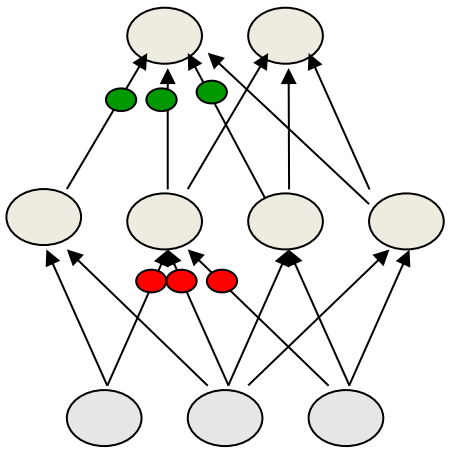
Algorithm 8.3 Stochastic gradient descent (SGD) with Nesterov momentum

Require: Learning rate ϵ, momentum parameter α.
Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$.
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding labels $\boldsymbol{y}^{(i)}$.
    Apply interim update: $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha \boldsymbol{v}$
    Compute gradient (at interim point): $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$
    Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
  end while

Adaptive learning rates

Adaptive learning rate

- In a multilayer net, the appropriate learning rates can vary widely between weights:
  - The magnitudes of the gradients are often very different for different layers, especially if the initial weights are small.
  - The fan-in of a unit determines the size of the "overshoot" effects caused by simultaneously changing many of the incoming weights of a unit to correct the same error.
- So use a global learning rate (set by hand) multiplied by an appropriate local gain that is determined empirically for each weight.



Gradients can get very small in the early layers of very deep nets.

The fan-in often varies widely between layers.

Determine the individual learning rates

- Start with a local gain of 1 for every weight.
- Increase the local gain if the gradient for that weight does not change sign.
- Use small additive increases and multiplicative decreases (for mini-batch)
  - This ensures that big gains decay rapidly when oscillations start.
  - If the gradient is totally random the gain will hover around 1 when we increase by plus δ half the time and decrease $1-\delta$ by times half the time.

$$\Delta w_{ij} = -\varepsilon\, g_{ij} \frac{\partial E}{\partial w_{ij}}$$

$$\text{if} \quad \left(\frac{\partial E}{\partial w_{ij}}(t) \frac{\partial E}{\partial w_{ij}}(t-1) \right) > 0$$

$$\text{then} \quad g_{ij}(t) = g_{ij}(t-1) + .05$$

$$\text{else} \quad g_{ij}(t) = g_{ij}(t-1) * .95$$

Tricks for adaptive learning rates

- Limit the gains to lie in some reasonable range
  - *e.g.* [0.1, 10] or [.01, 100]
- Use full batch learning or big mini-batches
  - This ensures that changes in the sign of the gradient are not mainly due to the sampling error of a mini-batch.
- Adaptive learning rates can be combined with momentum.
  - Use the agreement in sign between the current gradient for a weight and the velocity for that weight (Jacobs, 1989).
- Adaptive learning rates only deal with axis-aligned effects.
  - Momentum does not care about the alignment of the axes.

AdaGrad

- Definition: adapts the learning rates of all model parameters by scaling them inversely proportional to the square root of the sum of all of their historical squared values
- Results:
  - parameters with the largest partial derivative of the loss have a correspondingly rapid decrease in their learning rate,
  - parameters with small partial derivatives have a relatively small decrease in their learning rate

AdaGrad

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ

Require: Initial parameter $\boldsymbol{\theta}$

Require: Small constant δ, perhaps 10^{-7}, for numerical stability

Initialize gradient accumulation variable $\boldsymbol{r} = \mathbf{0}$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.

Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$

Accumulate squared gradient: $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$

Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$. (Division and square root applied element-wise)

Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$

end while

Root Mean Square Propagation (RMSProp)

Divide the gradient by a running average of its recent magnitude

Motivation

- AdaGrad shrinks the learning rate according to the entire history of the squared gradient and may have made the learning rate too small before arriving at such a convex structure
- RMSProp uses an exponentially decaying average to discard history from the extreme past

Root Mean Square Propagation (RMSProp):

- RMSProp: Keep a moving average of the squared gradient for each weight

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ, decay rate ρ.
Require: Initial parameter $\boldsymbol{\theta}$
Require: Small constant δ, usually 10^{-6}, used to stabilize division by small numbers.
Initialize accumulation variables $\boldsymbol{r} = 0$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1 - \rho) \boldsymbol{g} \odot \boldsymbol{g}$
Compute parameter update: $\Delta \boldsymbol{\theta} = -\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}} \odot \boldsymbol{g}$. ($\frac{1}{\sqrt{\delta + \boldsymbol{r}}}$ applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$
end while

AdaGrad vs. RMSProp

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ
Require: Initial parameter $\boldsymbol{\theta}$
Require: Small constant δ, perhaps 10^{-7}, for numerical stability
Initialize gradient accumulation variable $\boldsymbol{r} = \mathbf{0}$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$
Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$. (Division and square root applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ, decay rate ρ.
Require: Initial parameter $\boldsymbol{\theta}$
Require: Small constant δ, usually 10^{-6}, used to stabilize division by small numbers.
Initialize accumulation variables $\boldsymbol{r} = 0$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \rho\boldsymbol{r} + (1-\rho)\boldsymbol{g} \odot \boldsymbol{g}$
Compute parameter update: $\Delta\boldsymbol{\theta} = -\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}} \odot \boldsymbol{g}$. ($\frac{1}{\sqrt{\delta + \boldsymbol{r}}}$ applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Adam

- The name "Adam" derives from the phrase "adaptive moments."
- Adam: apply momentum to the rescaled gradients based on RMSProp

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)
Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$. (Suggested defaults: 0.9 and 0.999 respectively)
Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})
Require: Initial parameters $\boldsymbol{\theta}$
  Initialize 1st and 2nd moment variables $\boldsymbol{s} = \mathbf{0}$, $\boldsymbol{r} = \mathbf{0}$
  Initialize time step $t = 0$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    $t \leftarrow t + 1$
    Update biased first moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1)\boldsymbol{g}$
    Update biased second moment estimate: $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2)\boldsymbol{g} \odot \boldsymbol{g}$
    Correct bias in first moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}$
    Correct bias in second moment: $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t}$
    Compute update: $\Delta\boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (operations applied element-wise)
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
  end while

Summary

- Learning error
- Stochastic gradient descent
- Mini batch gradient descent
- Speeding up mini-batch gradient descent
  - Momentum
  - AdaGrad
  - Rmsprop
  - Adam

Courtesy

- Courtesy: Geoffrey Hinton, Nitish Srivastava , Kevin Swersky
- Dr. Hafiz Khan @ Kennesaw State University